

Serenity Harris

Professor Legand Burge

CSCI 375 Software Engineering

19 February 2026

Code Review Summary Report

The assignment was to develop a game called Boggle in python based on rules and specific criteria such as finding as many words in an NxN grid as possible, each word may not use a cube more than once, words must be at least 3 letters long, there should be no tiles in the NxN grid that contains “S”, “I” or “Q” tiles, and the only 2 letter tiles should be “Qu”, “St”, and “Ie.” Once this step is completed the next assignment was to develop unit tests for our code using black box testing. Lastly, we were asked to push our code to a GitHub repository where two partners would conduct code reviews on each other's code then merge the code into the main branch.

Upon reviewing my code a peer in class provided me with very helpful feedback. Specifically, my peer praised stopping recursion by placing an if statement that determines whether or not a word is a valid prefix at the beginning of the function. Additionally, my peer gave constructive criticism and explained that instead of checking if every row is the same length, I could use `len(self.grid[0])` for checking instead. After reviewing her code, I saw a similar positive that she had cut off her depth first search the moment a prefix does not match any known prefixes, which optimizes her code and helps with time and space complexity. One thing that might have needed improvement is moving the dictionary processing to `__init__` instead of building the dictionary of valid words in the `getSolution()` method. I believe this needs

improvement because if a user were to call this method multiple times on the same object, it would recreate the prefix hash each time despite not needing to.

Ultimately, I decided not to implement the recommendation given to me by my reviewer because I felt that checking if every row was equal to the `len(self.grid)` was a more thorough check and allowed me to check for all edge cases. I used the prettier extension in VSCode to refactor my code and make sure there were not any styling issues. Then I ran my code through the unit tests I had previously developed in order to do regression testing and make sure that changes in styling did not affect the functionality of the code. Through completing this project I learned a lot about code reviews.

I encountered many issues while trying to do this code review. The partner who was initially assigned to me did not review my code after I added him as a reviewer. Then, after I communicated with him in person he still did not review my code. I eventually communicated with the professor who sent an email to my partner. However, my partner did not respond to this email, so the professor asked the TA to review my code, but there was a problem with GitHub and the TA was not able to gain access to my pull request. Therefore, I talked with a peer, and they reviewed my code.

This sequence of events taught me a lot about communication in a semi-professional workspace. I learned to be prompt when communicating and to ensure that I receive timely responses. This ensures that I will be able to complete all of my assignments on time. Additionally, I learned the importance of having another person review your code because my reviewer was able to bring up a point that I had not previously considered and help me think about my code differently. Overall, I would say this experience taught me a lot.